# Stack und Heap

Wo liegen Variablen und Objekte im Speicher? – allgemein, in Java und in Python

## 1. Zwei Speicherbereiche für zwei Aufgaben

Ein laufendes Programm braucht Speicher für zwei sehr verschiedene Dinge:

- **Kurzlebiges**: lokale Variablen und Parameter einer Funktion. Sie werden nur gebraucht, solange die Funktion läuft. → **Stack**
- **Langlebiges**: Daten, die eine Funktion überleben sollen oder deren Größe erst zur Laufzeit feststeht (Listen, Objekte, Strings). → **Heap**

## 2. Der Stack (Stapel)

- Bei jedem Funktionsaufruf wird ein **Aufrufrahmen** (Stack Frame) oben draufgelegt. Er enthält Parameter, lokale Variablen und die Rücksprungadresse.
- Beim `return` wird der oberste Rahmen wieder entfernt – **automatisch**, ohne Aufräumarbeit. Prinzip: **LIFO** (last in, first out).
- Sehr **schnell**, weil nur ein Zeiger (Stack Pointer) verschoben wird.
- Relativ **klein** (typisch einige MB). Zu tiefe Rekursion → **Stack Overflow** (Java: `StackOverflowError`). Python bricht vorher kontrolliert mit `RecursionError` ab – Standardlimit 1000 Ebenen (`sys.getrecursionlimit()`).

```
main()            ruft auf ->  berechne()  ruft auf ->  hilfsfunktion()

Stack:  | hilfsfunktion |  <- oben: läuft gerade
        | berechne      |  <- wartet
        | main          |  <- wartet
```

## 3. Der Heap (Halde)

- Ein großer Speicherbereich, aus dem zur Laufzeit beliebig Platz **angefordert** wird (Java/Python: bei jedem neuen Objekt).
- Objekte leben so lange, wie sie noch **gebraucht** werden – unabhängig davon, welche Funktion sie erzeugt hat.
- Aufräumen ist die Schwierigkeit: In C macht man es von Hand (`malloc`/`free`), in Java und Python erledigt das ein **Garbage Collector**.
- Langsamer als der Stack (Verwaltung freier Bereiche, Aufräumen).

| | Stack | Heap |
|---|---|---|
| Inhalt | Aufrufrahmen: lokale Variablen, Parameter | Objekte |
| Lebensdauer | bis zum Ende der Funktion | solange noch eine Referenz existiert |
| Verwaltung | automatisch (LIFO) | Garbage Collector (Java, Python) |
| Geschwindigkeit | sehr schnell | langsamer |
| Größe | klein, fest begrenzt | groß |
| Fehler bei Überlauf | Stack Overflow | `OutOfMemoryError` / `MemoryError` |

## 4. Bekanntes Terrain: Java

- **Primitive Werte** (`int`, `double`, `boolean` …) liegen **direkt im Stack Frame**: `int zahl = 5;` → die 5 steht im Frame.
- Bei Objekten liegt im Frame nur die **Referenz**; das Objekt selbst liegt im **Heap**: `String name = "Anna";` → Referenz im Frame, String im Heap.

## 5. Und in Python?

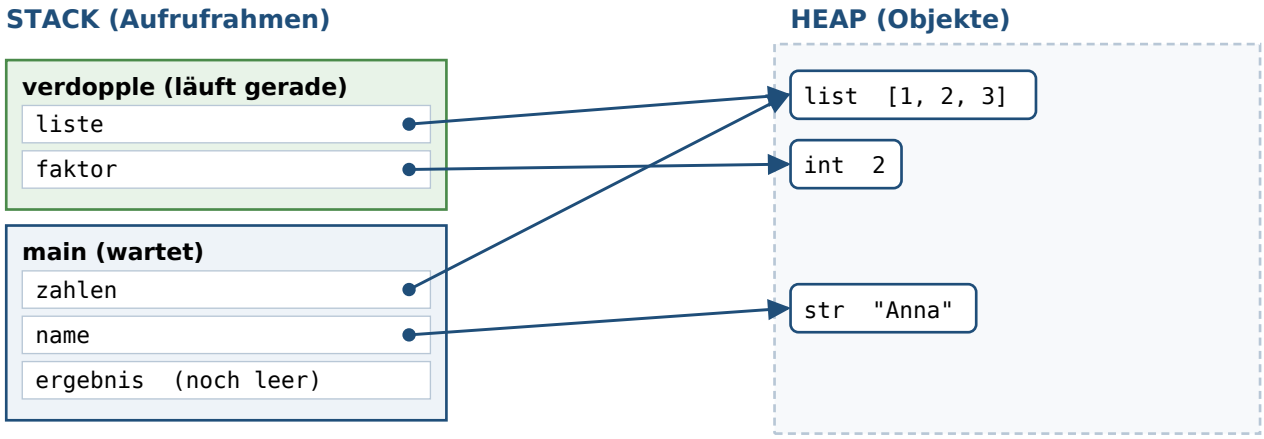
Python kennt keine primitiven Typen. Daraus folgt eine einfache Regel:

> **Im Stack Frame stehen nur Namen mit Referenzen. Alle Objekte – auch jede Zahl – liegen im Heap.**

```
def verdopple(liste):
    faktor = 2
    return [x * faktor for x in liste]      # <- Momentaufnahme hier

def main():
    zahlen = [1, 2, 3]
    name = "Anna"
    ergebnis = verdopple(zahlen)

main()
```




- `zahlen` (in `main`) und `liste` (in `verdopple`) zeigen auf **dieselbe** Liste. Beim Aufruf wird nur die Referenz übergeben, nicht die Liste kopiert.
- Sogar `faktor = 2` ist ein Objekt im Heap – in Java stünde die 2 direkt im Frame.
- Nach `return` verschwindet der Rahmen von `verdopple`. Die Liste bleibt, weil `zahlen` noch darauf zeigt.
- Vereinfachtes Modell: CPython verwaltet seine Frames intern etwas anders als ein C- oder Java-Programm. Für das Verständnis von Python-Code ist dieses Bild aber genau das richtige.

## 6. Wer räumt den Heap auf?

CPython benutzt zwei Mechanismen:

- **Referenzzählung:** Jedes Objekt zählt, wie viele Referenzen auf es zeigen. Fällt der Zähler auf 0, wird es **sofort** freigegeben.
- **Zyklen-Collector:** Findet Objekte, die nur noch aufeinander zeigen (z. B. eine Liste, die sich selbst enthält) und deren Zähler deshalb nie 0 wird.

```
import sys
x = [1, 2]
print(sys.getrefcount(x))   # 2  (x + der Parameter von getrefcount selbst)
y = x
print(sys.getrefcount(x))   # 3
del y                       # löscht nur den NAMEN y, nicht das Objekt
print(sys.getrefcount(x))   # 2
```

- **Achtung:** `del` löscht einen Namen, kein Objekt. Das Objekt verschwindet erst, wenn keine Referenz mehr darauf zeigt. Im Unterschied zu Java passiert das in CPython meist sofort und nicht erst irgendwann.